TaskFlow

Personal Task Manager

README · Educational Project · Gamaliel (Gammii90210)

Repo	github.com/Gammii90210/My-Taskflow-
Live URL	https://Gammii90210.github.io/My-Taskflow-/
Tech	HTML · CSS · Vanilla JavaScript — zero frameworks, zero npm
Storage	localStorage (web) · chrome.storage.sync (extension)
Author	Gamaliel (Gammii90210) · Educational purposes

> This project was built to teach front-end fundamentals without any frameworks. Everything runs in the browser — no backend, no build tools, no npm required.

How to Run the Project

There are four ways to run TaskFlow locally. Method 1 requires nothing — just a browser.

Method 1 — Open Directly in Browser (Fastest, zero setup)

> Requires: A web browser only. Nothing to install.

STEP 1 — Download the project

Go to: https://github.com/Gammii90210/My-Taskflow-
Click the green "Code" button → choose "Download ZIP"
Extract (unzip) the downloaded file anywhere on your computer

STEP 2 — Open the app

Open the extracted folder — you will see these 3 files:

```
index.html style.css app.js
```

Double-click index.html — TaskFlow opens immediately in your browser

> ■ All three files must stay in the same folder. Moving only index.html will break the styling and all JavaScript logic.

Method 2 — VS Code + Live Server (Best for development)

> Requires: VS Code + the free "Live Server" extension. Auto-reloads on every save.

STEP 1 — Install VS Code

Download from: https://code.visualstudio.com → Install and open it

STEP 2 — Install Live Server extension

In VS Code press Ctrl+Shift+X to open Extensions
Search for: Live Server by Ritwick Dey → click Install

STEP 3 — Clone the repo

Open VS Code Terminal (Ctrl + `) and run:

```
git clone https://github.com/Gammii90210/My-Taskflow-.git
```

```
cd My-Taskflow-
```

```
code .
```

STEP 4 — Launch

In the VS Code file explorer, right-click index.html
Click "Open with Live Server"
Browser opens at: http://127.0.0.1:5500 — auto-refreshes on every save

Method 3 — Python HTTP Server

> Requires: Python 3 — check with: python --version

```
git clone https://github.com/Gammii90210/My-Taskflow-.git
cd My-Taskflow-
python -m http.server 8000
# Then open: http://localhost:8000
```

Method 4 — Node.js http-server

> Requires: Node.js from https://nodejs.org

```
npm install -g http-server
cd My-Taskflow-
http-server
# Then open: http://localhost:8080
```

Chrome Extension Setup

TaskFlow also comes as a Chrome Extension — tasks sync across all your browsers via your Google account.

STEP 1 — Get the extension files

Same download as above — extract the ZIP, find the taskflow-ext/ folder inside

STEP 2 — Open Chrome Extensions

Open Google Chrome → type this in the address bar and press Enter:

```
chrome://extensions
```

STEP 3 — Enable Developer Mode

Toggle ON the "Developer mode" switch in the top-right corner — it turns blue

STEP 4 — Load the extension

Click "Load unpacked" → select the taskflow-ext/ folder → click Select Folder
TaskFlow appears in the list with the blue T icon

STEP 5 — Pin it to your toolbar

Click the ■ puzzle icon in Chrome toolbar → find TaskFlow → click the pin ■
Click the blue T icon anytime to open your tasks as a popup

	Web App	Chrome Extension
Storage	localStorage	chrome.storage.sync
Syncs devices	No	Yes — via Google account
Opens from	Browser tab	Toolbar popup
Offline	Yes	Yes
All features	Yes	Yes

File Structure

```
My-Taskflow-/
■
■■■ index.html ← App HTML structure (layout + skeleton)
■■■ style.css ← All visual styling (design system + animations)
■■■ app.js ← All JavaScript logic (state management + DOM)
■
■■■ taskflow-ext/ ← Chrome Extension version
■  ■■■ manifest.json Manifest V3 config
■  ■■■ popup.html Extension popup (400px wide)
■  ■■■ popup.css Extension styles
■  ■■■ popup.js Uses chrome.storage.sync
■  ■■■ icons/ PNG icons: 16 · 32 · 48 · 128px
■
■■■ README.md ← This document
```

Features

Feature	What it does
Create Tasks	Title + optional description + priority (High / Medium / Low)
Complete Tasks	Animated CSS checkbox — click to toggle done / undone
Edit Tasks	Inline — pencil icon replaces card content with input fields
Delete Tasks	Animated 220ms slide-out then removed from array
Filter	All / Active / Completed tabs — filters the visible list
Live Search	Real-time filter by title and description as you type
Progress Bar	Animated fill bar showing % of tasks completed

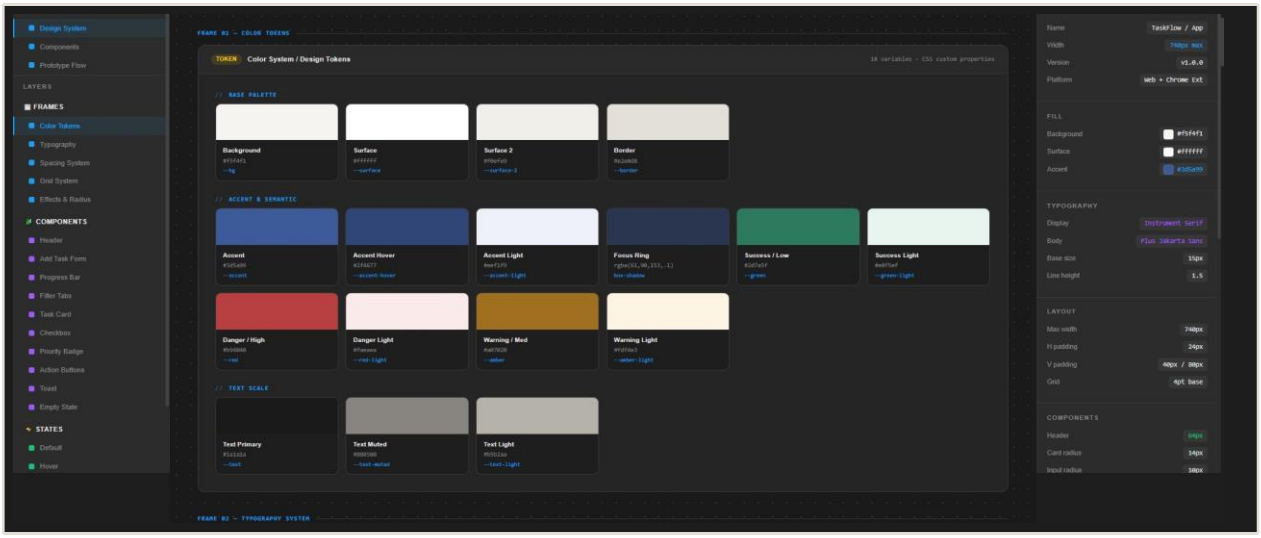
Stats Header	Live counters: Total · Active · Done — always current
Toast Alerts	Auto-dismissing feedback for every action (2.6 seconds)
Persistence	Tasks survive page refresh — saved to localStorage
Keyboard	Enter to add · Escape to cancel edit · Space on checkbox
Responsive	Mobile layout at 600px breakpoint

UI Design System — Figma Breakdown

TaskFlow uses a structured design token system — every color, size, and animation is a named variable. The screenshots below are from the interactive Figma-style breakdown built alongside the app.

Frame 01 — Color Tokens

18 CSS custom properties defined on :root in style.css. Changing --accent updates every button, tab, progress bar, and focus ring at once.

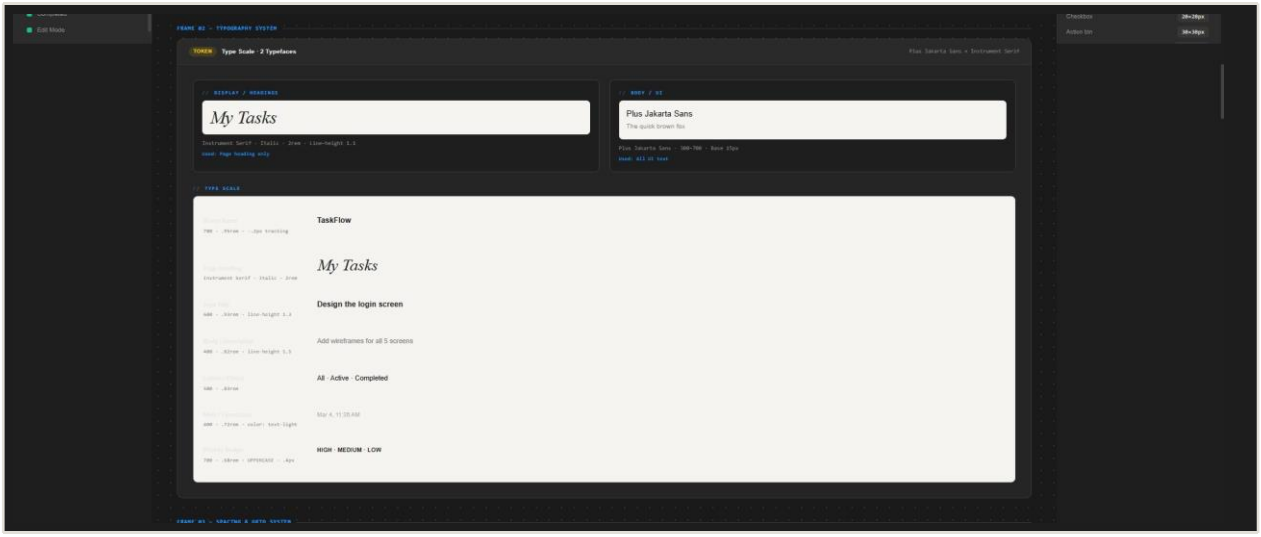


Frame 01 — Color System / Base palette · Accent & Semantic · Text scale

Token	Hex	Used For
--accent	#3d5a99	Buttons, active tab, progress bar, focus borders
--green	#2d7a5f	Checkbox checked, success toast, Low priority stripe
--red	#b94040	Delete hover, High priority stripe, error toast
--amber	#a07020	Medium priority stripe and badge
--bg	#f5f4f1	Page background — warm off-white
--surface	#ffffff	Cards, forms, header background
--text	#1a1a1a	Primary text — near black
--text-muted	#888580	Labels, descriptions, meta info

Frame 02 — Typography System

Two typefaces only. Instrument Serif Italic for the page heading — Plus Jakarta Sans for everything else.

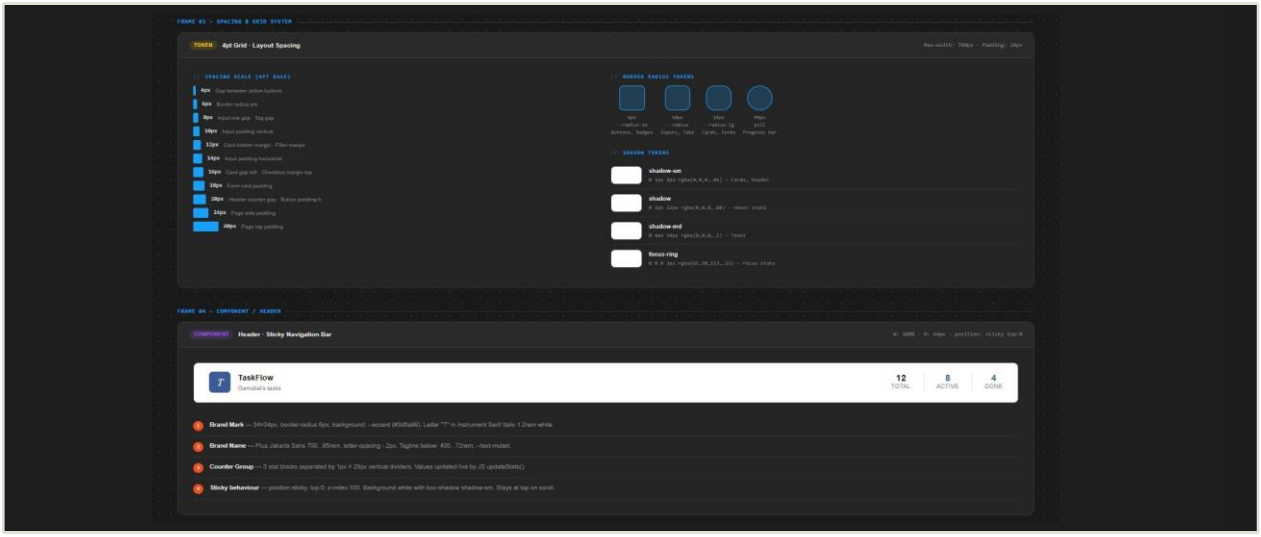


Frame 02 — Type scale · Display heading (Instrument Serif) · Body/UI (Plus Jakarta Sans)

Style	Font	Weight / Size
Page Heading	Instrument Serif	Italic 400 · 2rem
Task Title	Plus Jakarta Sans	600 · .93rem · line-height 1.3
Body / Description	Plus Jakarta Sans	400 · .82rem · line-height 1.5
Priority Badge	Plus Jakarta Sans	700 · .68rem · UPPERCASE
Timestamp	Plus Jakarta Sans	400 · .72rem · color: --text-light

Frame 03 — Spacing & Grid

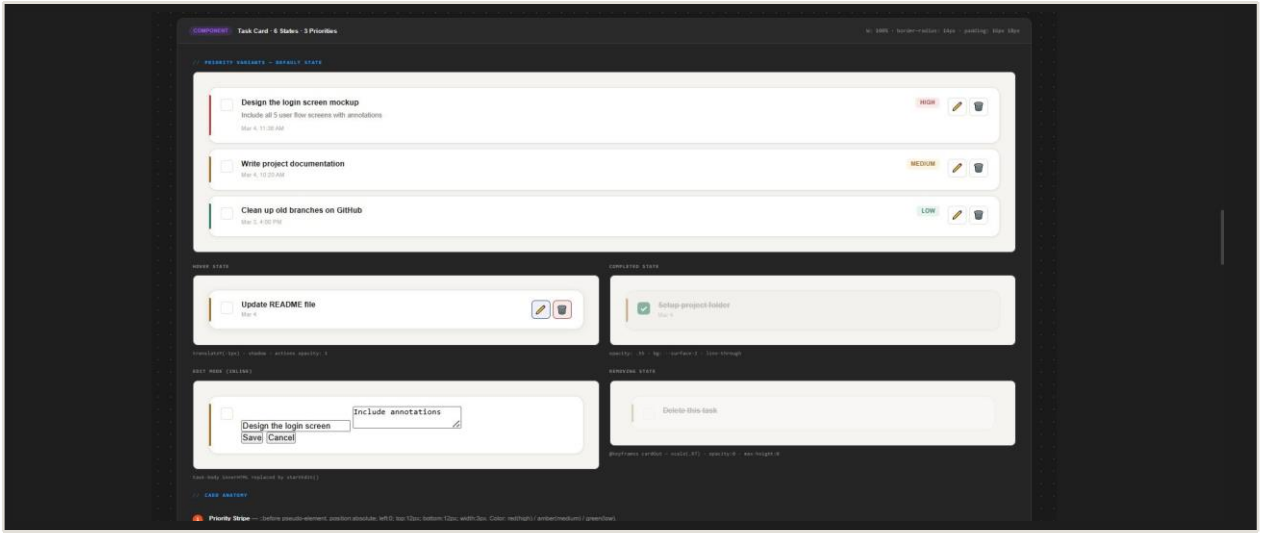
4pt base grid. All spacing values are multiples of 4. Border radius and shadows are also tokenised.



Frame 03 — 4pt spacing scale · Border radius tokens (6/10/14/99px) · Shadow tokens

Frame 06 — Task Card · 6 States · 3 Priorities

The most complex component. Priority stripe uses ::before pseudo-element. Action buttons are hidden by default, revealed on hover via CSS opacity.



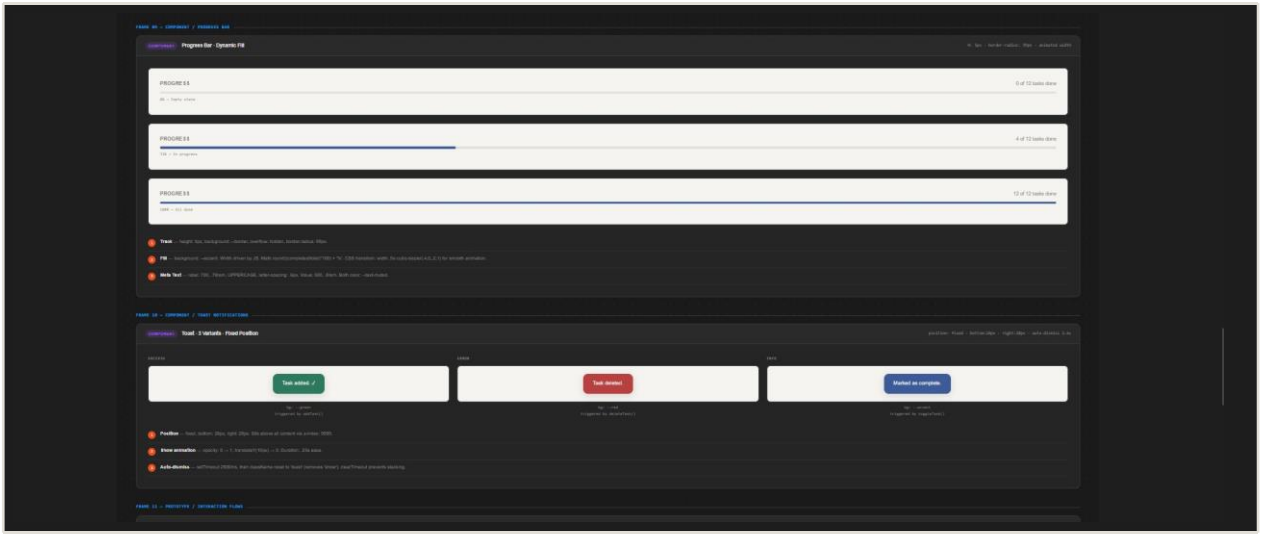
Frame 06 — Priority variants (high/medium/low) · Hover · Completed · Edit mode (inline) · Removing animation

State	How it looks	What triggers it
Default	White card, actions hidden (opacity:0)	Resting
Hover	translateY(-1px), shadow, actions visible	Cursor hover
Completed	opacity:.55, strikethrough title	.completed class
Edit Mode	Inputs replace card body	startEdit() function
Removing	scale(.97) + fade + collapse 220ms	.sremoving + @keyframes

Priority stripe: CSS ::before — position:absolute; left:0; width:3px; top/bottom:12px. Red = high · Amber = medium · Green = low.

Checkbox tick: Pure CSS — border right+bottom, rotated 45°. No images.

Frame 09 — Progress Bar + Frame 10 — Toast Notifications

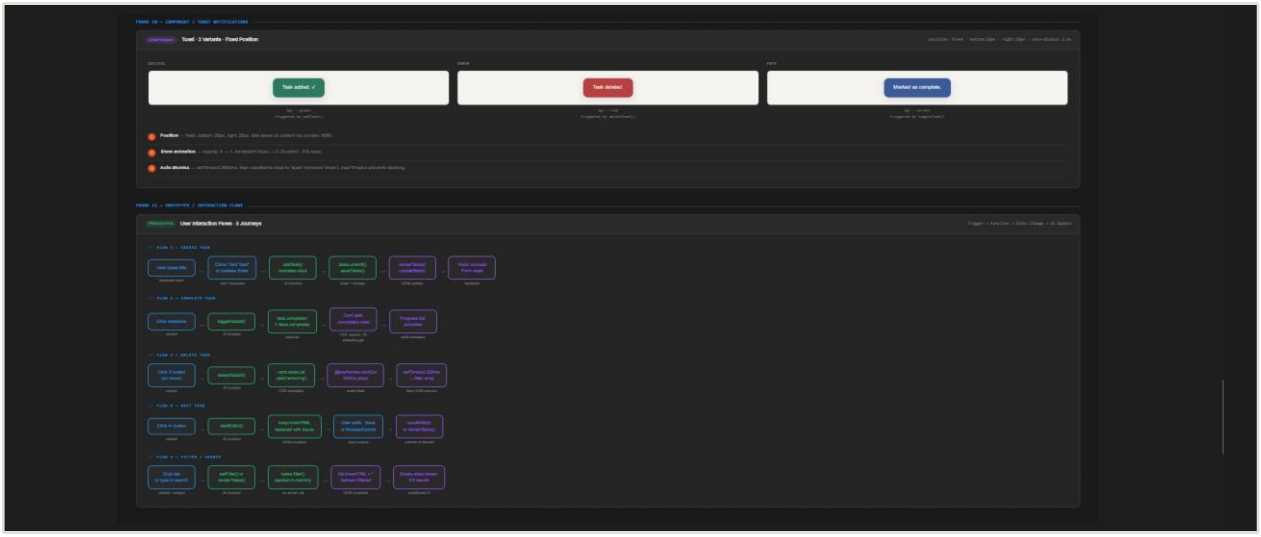


Frame 09/10 — Progress bar at 0% · 33% · 100% · Toast variants: success (green) · error (red) · info (blue)

Component	Key detail
Progress track	height: 5px · background: --border · overflow: hidden
Progress fill	background: --accent · width from JS · transition .5s cubic-bezier
Toast position	fixed bottom:28px right:28px · z-index:9999
Toast animation	translateY(10px)→0 · opacity 0→1 · .25s ease
Toast dismiss	setTimeout(2600ms) removes .show class · clearTimeout prevents stacking

Frame 11 — Interaction Flows

All 5 user journeys mapped: Create · Complete · Delete · Edit · Filter+Search



Frame 11 — 5 user interaction flows: each shows Trigger → JS function → State change → DOM update → Feedback

Interaction Flows — Detail

Create Task

```
User types title → clicks Add Task (or Enter)
addTask() validates →
tasks.unshift({id,title,desc,priority,completed:false,createdAt})
saveTasks() → localStorage → renderTasks() → updateStats()
Toast: "Task added." (green) · form reset
```

Complete Task

```
Click checkbox → toggleTask(id)
task.completed = !task.completed → saveTasks()
Card gets .completed class → opacity:.55 + strikethrough (CSS)
Progress bar animates to new %
```

Delete Task

```
Hover card → action buttons appear → click trash icon
deleteTask(id) → card.classList.add("removing")
@keyframes cardOut plays 220ms → setTimeout(220ms)
tasks.filter(t => t.id !== id) → saveTasks() → renderTasks()
```

Edit Task (inline)

```
Hover card → click pencil icon → startEdit(id)
task-body innerHTML replaced with input + textarea + Save/Cancel
Save → saveEdit(id) → task.title = new · task.editedAt = now → save + render
Cancel / Escape → renderTasks() — discards changes
```

Filter + Search

```
Click tab or type in search box
renderTasks() applies: skip completed if filter=active, skip active if
filter=completed
skip tasks not matching search query (title or desc)
0 results → empty state shown
```

JavaScript Functions

Function	What it does
addTask()	Validates input, creates task, prepends to array, saves, renders
toggleTask(id)	Flips completed boolean, saves, re-renders
deleteTask(id)	Triggers exit animation, waits 220ms, filters array, re-renders
startEdit(id)	Replaces card body with inline edit inputs

saveEdit(id)	Commits edits, sets editedAt timestamp, saves, re-renders
setFilter(f, btn)	Updates currentFilter, updates tab styles, re-renders
clearCompleted()	Removes all completed tasks in bulk
renderTasks()	Applies filter + search, rebuilds entire task list DOM
updateStats()	Recalculates totals, updates counters and progress bar
saveTasks()	JSON.stringify(tasks) → localStorage.setItem
loadTasks()	localStorage.getItem → JSON.parse back to tasks array
showToast(msg, type)	Shows notification, auto-dismisses after 2600ms
escapeHtml(str)	Sanitises user input — prevents XSS before innerHTML insert

Data Model

Each task is a plain JavaScript object. The full array is serialised to JSON and stored in localStorage under the key **taskflow_v1**.

```
{
id: "lf4k2abc9", // base36 timestamp + random string
title: "Design the UI", // Required. Max 120 chars
desc: "Include all flows", // Optional. Free text
priority: "high", // "high" | "medium" | "low"
completed: false, // Boolean — toggled by checkbox
createdAt: "2026-03-04T11:36:00Z", // ISO 8601 — set on creation
editedAt: null // null until first edit
}
```

Educational Notes

Key front-end concepts this project teaches:

1. CSS custom properties as a design token system — changing --accent in one place updates every button, tab, progress bar across the whole app. Mirrors how Figma variables work.
2. :focus-within CSS pseudo-class — the add form card highlights when any child input is focused. Pure CSS, no JavaScript needed.
3. Pure CSS checkbox tick — the checkmark is drawn with rotated borders. No images or SVG icons.
4. Animation timing coordination — the delete @keyframes plays 220ms, then JS waits exactly 220ms via setTimeout before removing the element.
5. XSS prevention — all user input goes through escapeHtml() before insertion via innerHTML, converting < > " ' & to safe entities.
6. Chrome Manifest V3 — inline onclick handlers are blocked by Content Security Policy, so all events use addEventListener in popup.js.